# TECHNOLOGICAL FUNDAMENTALS IN THE BIG DATA WORLD



BY NC SA
for the Alejandro Calderón Mateos work

# Goals



- Introducing the web service paradigm comparing with other common paradigms.
- Introducing the architecture of a web service.
- Knowledge of advanced technology for data processing.

# What is a Web Service ?



- A **web service** is a way for two applications to communicate over the internet using standard protocols.
  - They allow data exchange between different platforms (e.g., Python ↔ Java, Windows ↔ Linux).

- A Web service is an application that:
  - Runs on a Web server
  - It is platform and language independent
  - Exposes Web methods to interested callers
  - Listens for HTTP requests representing commands to invoke Web methods
  - Use **HTTP, XML, JSON** for communication
  - Executes Web methods and returns the results

# Web Services are based on:

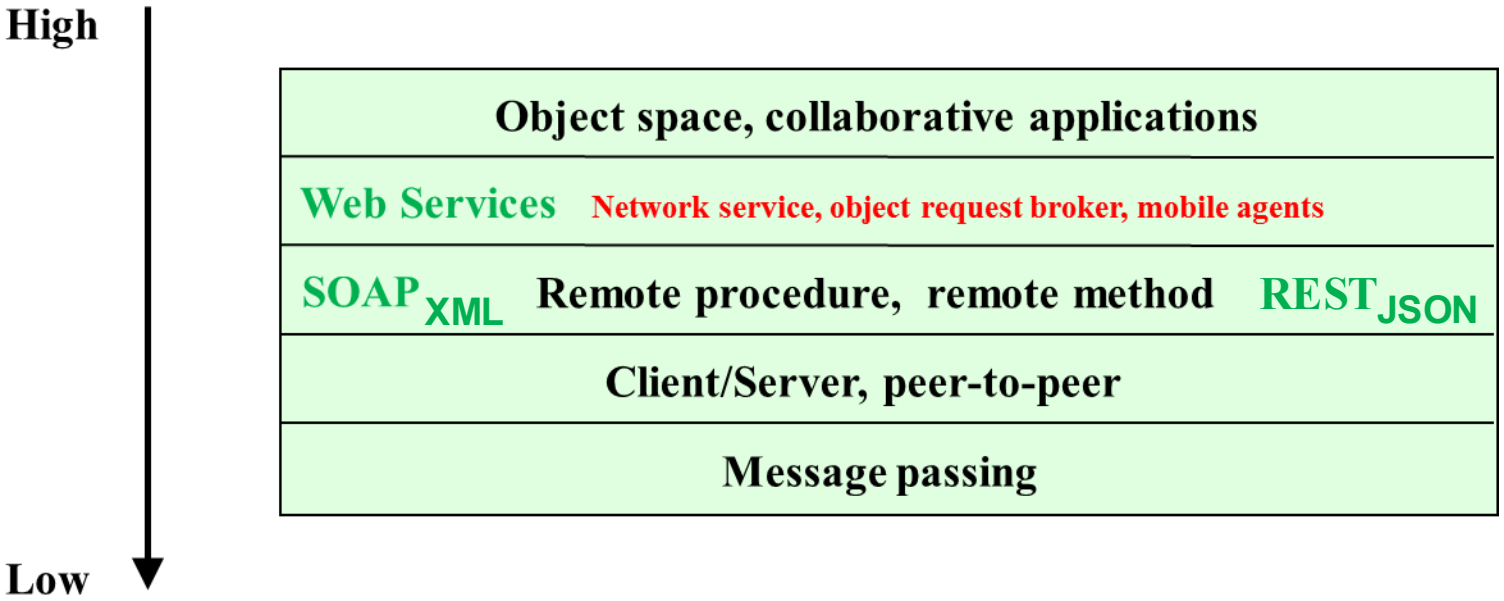


- Acronyms:
  - HTTP (Hypertext Transport Protocol)
  - SOAP (Simple Object Access Protocol)
  - UDDI (Universal Description, Discovery and Integration)
  - WS-POLICY (Web Services Policy)
  - WSDL (Web Service Description Language)

- Most Web services expect their Web methods to be invoked using HTTP requests containing SOAP messages.
  - SOAP is an XML-based vocabulary for performing remote procedure calls using HTTP and other protocols.

# Network services paradigm…

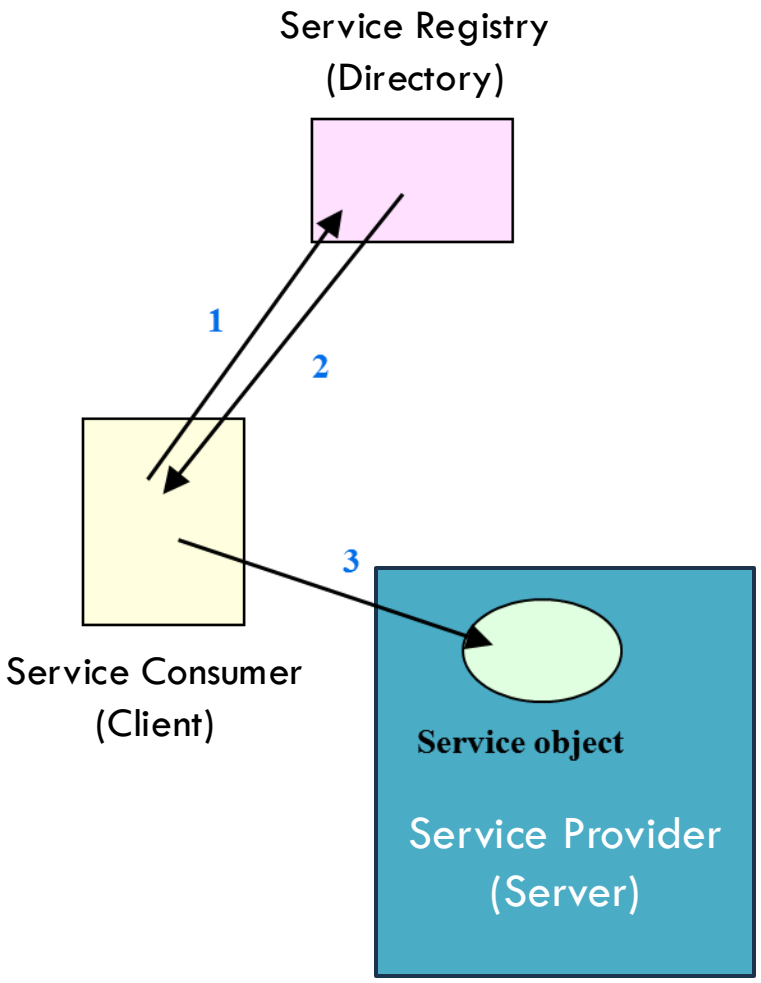





- It can be saw as an extension of remote invocation method (or remote procedure call).
- It adds location transparency: an extra abstraction level.

# Network services paradigm…

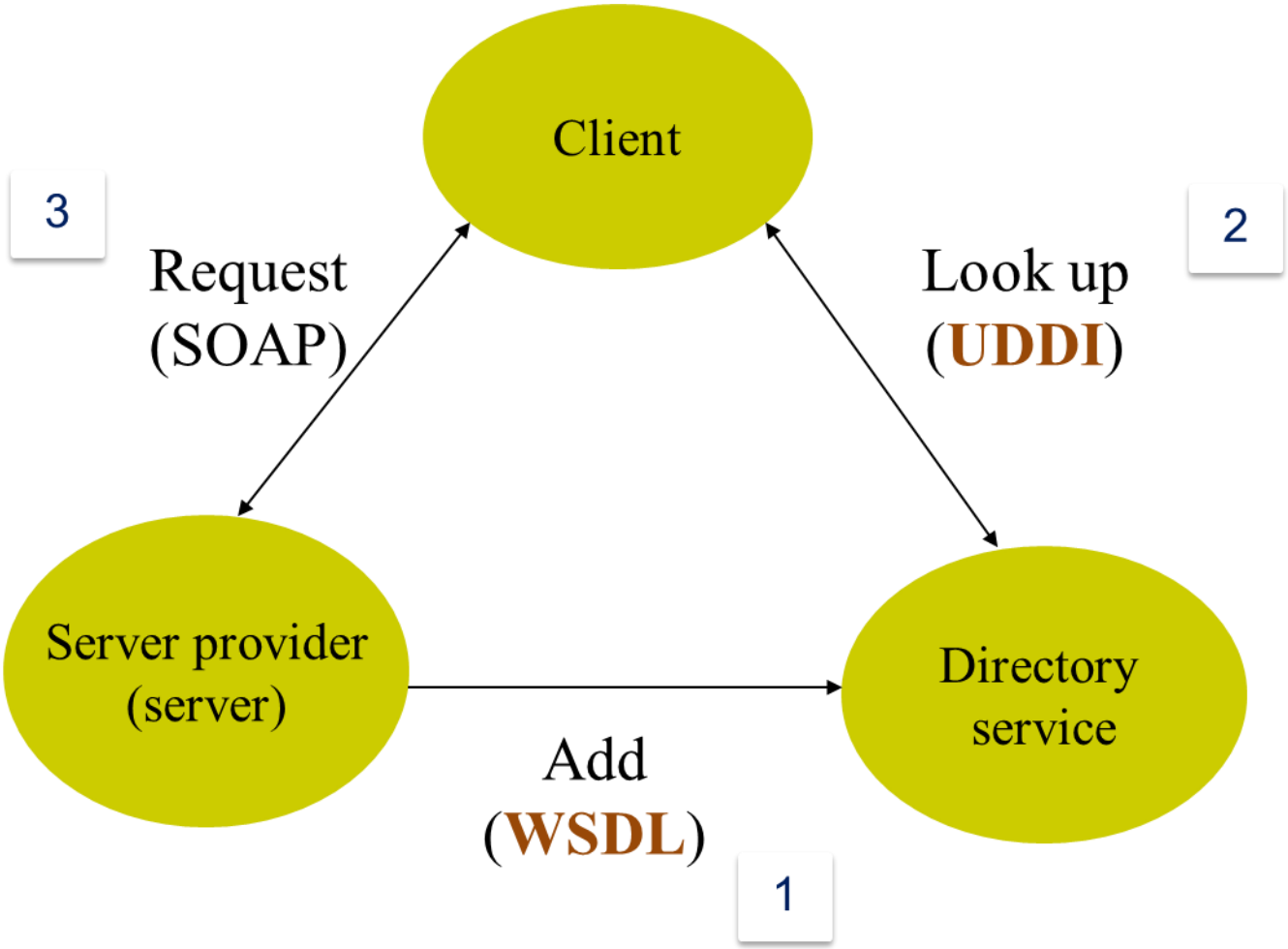





- A directory service provides the reference (abstract location) to the available services.
- Steps:
  1. Request process asks the directory service for a service.
  2. The directory service provides a reference to that service.
  3. By using the reference, the request process interacts with the service.
- Example: Java Jini

# Web Services and SOAP





# Web Services Description Language - WSDL



If other developers are to consume (that is, write clients for) a Web service that you author, they need to know :

- What Web methods your service publishes
- What protocols it supports
- The signatures of its methods
- The Web service's location (URL)

All this information and more can be expressed in a language called the Web Services Description Language, or WSDL for short.

- WSDL is an XML vocabulary
- http://www.w3.org/TR/wsdl.

# URI, URL, and URN



- Every Web Service has a URI (Uniform Resource Identifier).
  - Clients use the URI in order to reference the Web Service
  - Two types of URI:
    - URL (uniform resource locator)
      - It includes the resource location (*hostname+pathname*)
      - Example: ftp://ftp.funet.fi/rfc/rfc1808.txt
    - URN (uniform resource name)
      - Resource names that do not include location
      - Example: urn:isbn:0123456789
- There are a location service for services (directory):
  - URN -> URL
  - It enables location transparency

# WSDL



- **WSDL**: Web Services Description Language
  - IDL for Web Services in XML

- It is used for:
  - Describing Web Services
    - It specify the operations and the services methods
  - Looking up Web Services

- WSDL is a XML document
  - Written in XML
  - Standard described by W3C
    - http://www.w3.org/TR/wsdl
    - http://www.w3.org/TR/wsdl20

# WSDL document structure



```
<definitions>
    <types>
        type definition (language independent)
    </types>
    <message>
        message definition (to interchange)
    </message>
    <portType>
        port definition (function interface, including parameters, etc.)
    </portType>
    <binding>
        binding definition (message format and data to be used)
    </binding>
    <services>
        service definition (service's name and ports where is offered)
    </services>
</definitions>
```

# Name space



- Defined by W3C: http://www.w3.org/2001/XMLSchema
- Goal: avoid conflicts
- Example:
  - Two different Web Services A and B that have one common element f.
  - Every instance of f can be referred as A:f or B:f

# Web Service Discovery

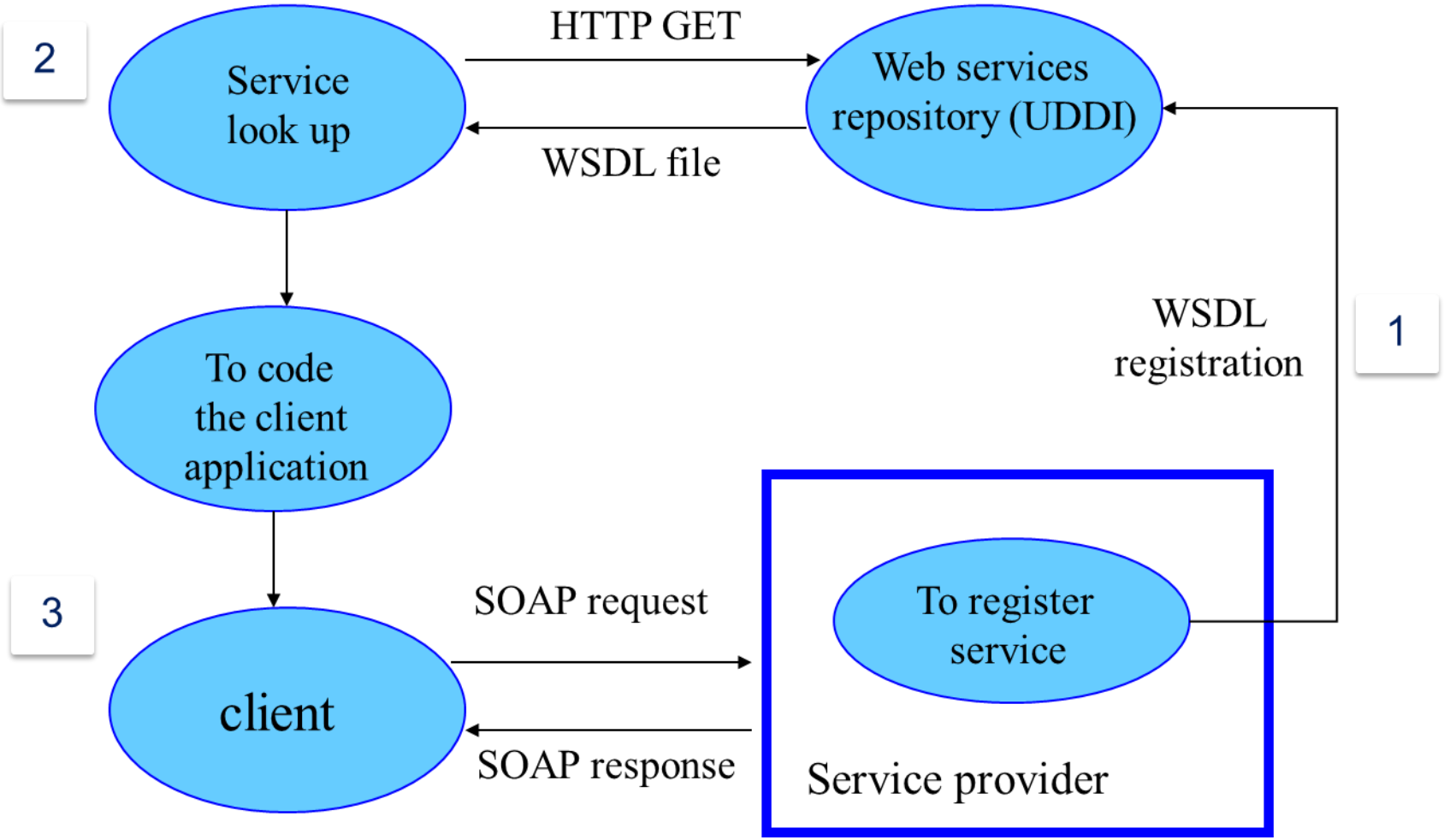


- **UDDI:** Universal Description, Discovery, and Integration
  - It enables businesses to publish service listings and discover each other

- Initial proposal by Microsoft, IBM and Ariba
- It enables to look up for different criteria:
  - Kind of activity, type of service, geographical localization, etc.
- Information classified in 3 categories (components):
  - White pages: information about the business supplying the service
  - Yellow pages: classification of service/business based on taxonomies
  - Green pages: technical information about services exposed

- It is used like another Web Service:
  - It can be used during the application development or dynamically on runtime

# Use case


2
Service look up
HTTP GET
WSDL file
Web services repository (UDDI)
WSDL registration
1
To code the client application
3
client
SOAP request
SOAP response
To register service
Service provider

# Types of Web Services



- **SOAP (Simple Object Access Protocol)**
  - Uses **XML** for requests & responses
  - Strict and standardized
  - Often used in enterprise systems

- **REST (Representational State Transfer)**
  - Uses **HTTP methods** (GET, POST, PUT, DELETE)
  - Lightweight, flexible, uses **JSON or XML**
  - Dominant in modern web apps

# JSON vs XML



- JSON:

```
{"menu": {
  "id": "file",
  "value": "File",
  "popup": {
     "menuitem": [
        {"value": "New", "onclick": "CreateNewDoc()"},
        {"value": "Open", "onclick": "OpenDoc()"},
        {"value": "Close", "onclick": "CloseDoc()"}
     ]
  }
}}
```

- XML:

```
<menu id="file" value="File">
   <popup>
        <menuitem value="New" onclick="CreateNewDoc()" />
        <menuitem value="Open" onclick="OpenDoc()" />
        <menuitem value="Close" onclick="CloseDoc()" />
   </popup>
</menu>
```

- *Simple Object Access Protocol*
  - http://*www.w3.org*

- SOAP specifies:
  - How XML messages are represented
  - How to combine SOAP messages in a request-response model
  - How to process the message elements
  - How to use the transport (HTTP, SMTP, …) in order to send SOAP messages

# REST Web Services



- REST means *Representational State Transfer*
- REST API is a standard
- Style for organizing the software architecture of distributed system based on HTTP
  - Part of the PhD thesis of Roy Fielding in 2000
  - Based on URL to locate resources and HTTP for resource management
- Used for:
  - Cloud applications
  - Cloud services
  - Web sites
- POSTMAN is a very popular place to test your services

# REST APIs benefits and challenges



- Benefits
  - No state to manage, as the web services are completely stateless.
  - Scalability, as client and servers are separated
  - Flexibility and portability, as the client gets the full answer without state information
  - Independence of client and provider for development.
  - Lightweight and fast, through HTTP and HTTPS, supporting multiple formats below (json, XML, HTML,..)
- Challenges:
  - REST endpoint consistency, especially in large codebases
  - REST API versioning, explosive due to legacy code
  - REST API authentication: open, login, registry
    - HTTP basic authentication (BA)
    - Token authentication (PHP)
    - Combining password and tokens (Keys, OAuth, …)

# REST API



- REST uses **HTTP methods:**
  - **GET** → Retrieve data
  - **POST** → Create data
  - **PUT** → Update data
  - **DELETE** → Delete data
- Example:
  - https://jsonplaceholder.typicode.com/posts/1
  - Protocol: https
  - Web server: jsonplaceholder.typicode.com
  - Method: posts
  - Arg: 1
  - Output ->

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati",
  "body": "quia et suscipit..."
}
```

# Python REST web services: request



- **Request** library to execute web calls
  - *request.get (URL);*
    - this is used to send a GET request to the URL.
  - *request.post (URL, data= dict);*
    - dict contains values to send to a POST request.
  - *request.put (URL, data =dict);*
    - this also works similarly to POST request.
  - *request.delete(URL, data =dict);*
    - dict sends the URL and values to the DELETE request method.

# Request examples: GET



- **Fetching** a blog post from a web service.(like a blog post or a user).

```
## Example 2
import requests
# Example: GET request to a public API (JSONPlaceholder)
response =
requests.get("https://jsonplaceholder.typicode.com/posts/1")
if response.status_code == 200:
    data = response.json()
    print(data)
```

```
## Example 1
import requests
url ="http://www.elpais.com"
print (url)
resp = requests.get(url)
print (resp.status_code)
print (resp.content)
```



```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati",
  "body": "quia et suscipit..."
}
```

# Request examples: POST



- **Creating a new resource** (like a blog post or a user).

```
import requests

url = "https://jsonplaceholder.typicode.com/posts"
new_post = {
    "title": "My API Post",
    "body": "This is sent via REST API",
    "userId": 1
}

response = requests.post(url, json=new_post)

print(response.status_code)    # 201 (Created)
print(response.json())
```

# Request examples: PUT



- **Updating an existing resource** (like a blog post or a user).

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"
update_post = {
    ”id” = 1,
    "title": "My API Put",
    "body": ”Updating post 1",
    "userId": 1
}

response = requests.put(url, json=update_post)

print(response.status_code)    # 200 (Updated)
print(response.json())
```

# Request examples: DELETE



- **Deleting an existing resource** (like a blog post or a user).

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.delete(url)

print(response.status_code)    # 200 (Deleted)
```

# PyCURL



- Tool for transferring data requests to and from a server.
- curl is a UNIX command which can send GET, POST, PUT and DELETE request to a URL.
- PyCURL is the library to use curl for testing REST APIs.

- More:
  - http://pycurl.io/docs/latest/quickstart.html

# PyCURL example



```
import pycurl
from io import BytesIO


b_obj = BytesIO()
crl = pycurl.Curl()
crl.setopt(crl.URL,'http://pycurl.io')
crl.setopt(crl.WRITEDATA, b_obj)
crl.perform()
crl.close()
get_body = b_obj.getvalue()
print("Output of GET request:",  get_body.decode("utf8"))
```

# Contents



- Introduction
  - Where Web Services are…
  - Evolution towards Web Services…
- XML + HTTP + SOAP
- UDDI + WSDL
- JSON + REST
- **Web Service as SDK.**

# FLASK



- Flask is a Python microframework used to build web applications and REST APIs.

# Flask example



```
# Flask Example. Ex7-flask.py
from flask import Flask, request, jsonify

app = Flask(__name__)

countries = [
    {"id": 1, "name": "Thailand", "capital": "Bangkok", "area":
513120},
    {"id": 2, "name": "Australia", "capital": "Canberra", "area":
7617930},
    {"id": 3, "name": "Egypt", "capital": "Cairo", "area": 1010408},
]

def __find_next_id():
    return max(country["id"] for country in countries) + 1

@app.get("/countries")
def get_countries():
    return jsonify(countries)
```

# Flask example



```
@app.post("/countries")
def add_country():
    if request.is_json:
        country = request.get_json()
        country["id"] = _find_next_id()
        countries.append(country)
        return country, 201
    return {"error": "Request must be JSON"}, 415
```

To setup the services in **LINUX**:

```
>>export FLASK_APP=ex7-flask.py
>>export FLASK_ENV=development
>>flask run
```

To setup the services in **Windows**

```
C:\> set FLASK_APP= ex7-flask.py
C:\> set FLASK_ENV=development
C:\> flask run
```

# Flask example. Access to ws



To acces the services, from the Shell:

- Get the countries

```
$curl -i http://127.0.0.1:5000/countries \
-X GET \
-H 'Content-Type: application/json'
```

- Create a new country

```
$ curl -i http://127.0.0.1:5000/countries \
-X POST \
-H 'Content-Type: application/json' \
-d '{"name":"Germany", "capital": "Berlin", "area": 357022}'
```

To acces the services, from the web browser:

- Get the countries:
  >> http://127.0.0.1:5000/countries

- See a Python country example for country creation. (ex8)

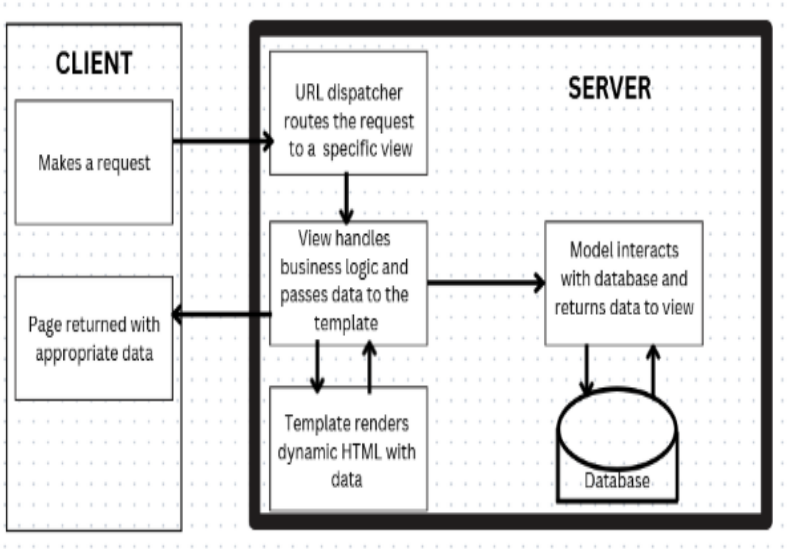
- Exercise: Modify the ex7 to include 2 web services to update the info of a Country by name and to delete a Country by name.

# Django REST Framework



- High-level Python framework. It's popular for its simplicity and efficiency in building robust web applications.
- The heart of Django's architecture is the Model-View-Template (MVT)

  - **Model**: Also known as the data layer, it manages data and interacts with the database.
  - **View**: Also known as the logic layer, it acts as the intermediary, handles logic, and manages data flow.
  - **Template**: Also known as the presentation layer, it renders HTML content on the user interface.



# Django REST Framework. Project creation



```
>>pip install django djangorestframework
>>mkdir django; cd django
```

- Start new project

```
>>django-admin startproject countryapi
```

- Move to project folder

```
>>cd countryapi
```

- Create the application

```
>>python manage.py startapp countries
```

- countryapi folder created with base files for country

```
>>ls countryapi
asgi.py settings.py urls.py wsgi.py
```

- countries folder created with base files for app

```
>>ls countries
admin.py apps.py migrations models.py tests.py views.py
```

# Django REST Framework. Configure the project



- Edit countryapi "settings.py" file to add app and rest framework

- In INSTALLED_APPS = [
- Add:
  - `"rest_framework",`
  - `"countries",`

- Apply migration to Django server

```
>> python manage.py migrate
```

- Project ready.
  - Now we have to setup the application.

# Define app database models



- Edit *countries/models.py* to define database table structure.

- Add:

```
class Country(models.Model):
    name = models.CharField(max_length=100)
    capital = models.CharField(max_length=100)
    area = models.IntegerField(help_text="(in square kilometers)")
```

- Update the project database

```
>> python manage.py makemigrations
>> python manage.py migrate
```

# Upload app db with initial data



- We have created *countries/countries.json* to load initial data
- See *countries/countries.json content*.

```
[
    {
        "model": "countries.country",
        "pk": 1,
        "fields": {
            "name": "Thailand",
            "capital": "Bangkok",
            "area": 513120
        }
    },
...
]
```

- Update the project database with data

```
>> python manage.py loaddata countries/countries.json
```

# Define app db data serializer



- Create *countries/serializers.py to say Django how to* convert JSON data for a REST API.
- Add:

```
 # countries/serializers.py
from rest_framework import serializers
from .models import Country

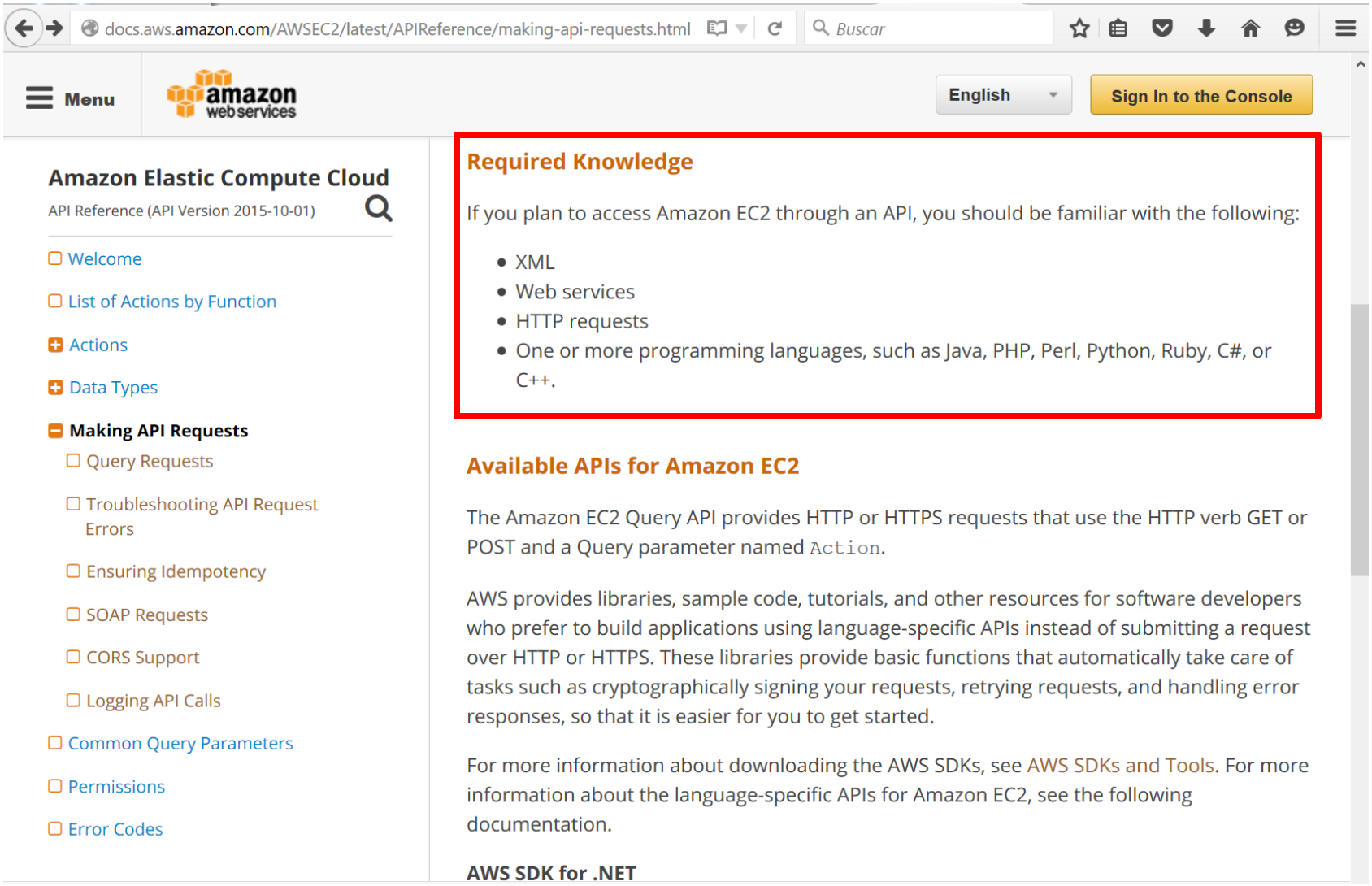
class CountrySerializer(serializers.ModelSerializer):
    class Meta:
        model = Country
        fields = ["id", "name", "capital", "area"]
```

- Update the project database with data

```
>> python manage.py loaddata
countries/countries.json
```

# Amazon Web Services





# Mapp views to URLs



- Create *countries/urls.py* to define database table structure.
- Add:

```
from django.urls import path,
include
from rest_framework.routers import
DefaultRouter

from .views import CountryViewSet

router = DefaultRouter()
router.register(r"countries",
CountryViewSet)

urlpatterns = [
    path("", include(router.urls))
]
```

- Update the Project base urls.py file
- Edit countryapi/urls.py as follows:

```
from django.contrib import
admin
from django.urls import path,
include

urlpatterns = [
    path("admin/",
admin.site.urls),
    path("",
include("countries.urls")),
]
```

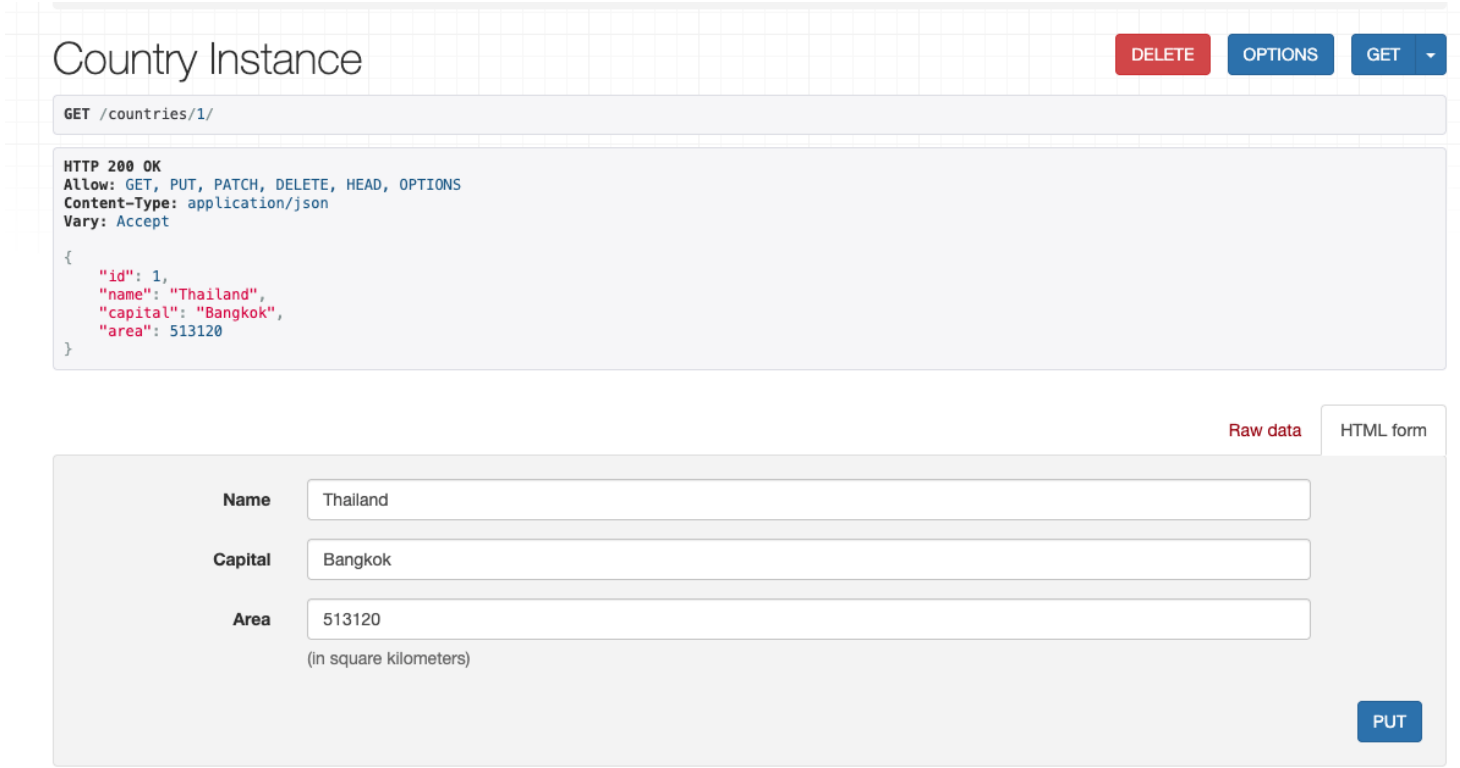
# Run the Django server and test



- Run the server

```
>> python manage.py runserver
```

- Test the app in browser:
  - `http://127.0.0.1:8000/countries/`



# Testing the service from Python



```
import requests


##  get db contents

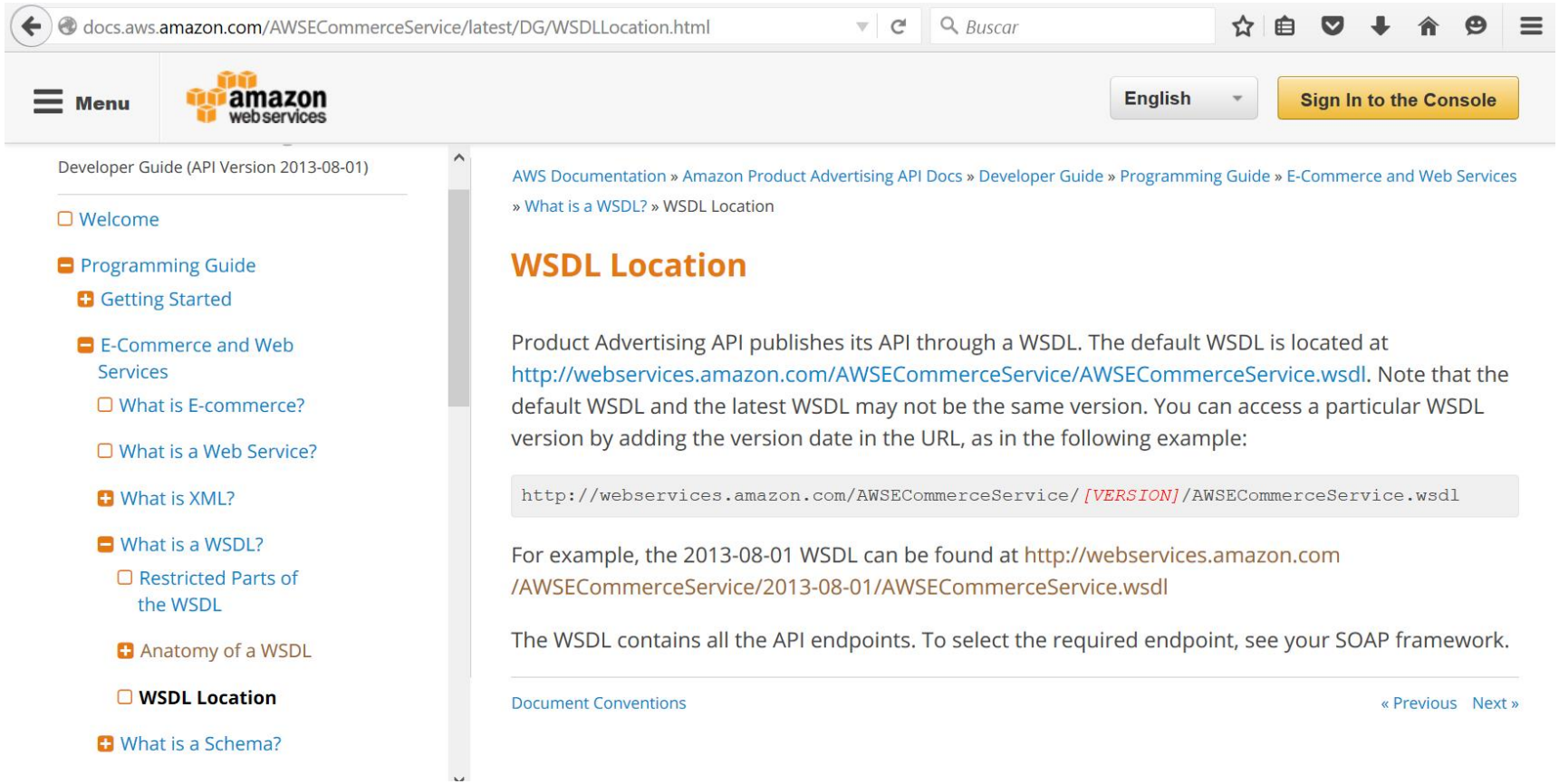
print ("########## GET ##################")
res=requests.get("http://127.0.0.1:8000/countries/")
print (res.text)
print (res)


print ("########## GET 3 ##################")
##  get db contents country 3
res=requests.get("http://127.0.0.1:8000/countries/3")
print (res.text)

print ("########## POST country ##################")
object = {"id":"10","name":"Mexico","capital":"Mexico","area":980765}
res=requests.post("http://127.0.0.1:8000/countries/",data=object)
print (res.text)
```

# Amazon Web Services





# SDK for AWS…



https://aws.amazon.com/es/documentation/

Buscar

Menú | amazon web services | Productos | Soluciones | Precios | Más | Español | Mi cuenta | Inscríbase

## Documentación de AWS

Bienvenido a la documentación de Amazon Web Services (AWS). Sea nuevo en AWS o un usuario avanzado, podrá encontrar información útil sobre los servicios que va desde introducciones hasta características avanzadas. Haga clic en las pestañas para ver las distintas categorías de servicios y herramientas de la nube de AWS, o consulte un vocablo en el glosario de AWS.



Servicios | Introducción | Desarrolladores

**Kits de desarrollo de software**

SDK de AWS para Java
AWS SDK para JavaScript
AWS SDK para .NET
SDK de AWS para PHP
AWS SDK para Python (boto)
AWS SDK para Ruby

**Servicios móviles y SDK**

**Herramientas**

AWS Toolkit for Eclipse
AWS Toolkit for Visual Studio
Interfaz de línea de comandos de AWS
Herramientas de AWS para Windows PowerShell

**Herramientas para desarrolladores**

**AWS Management Console**

Resource Groups
Tag Editor

**Referencia general**

Regiones y extremos
Credenciales de seguridad
Firma de solicitudes API
Espacios de nombres del

**Recursos**

Gestión de costes y facturación de AWS
AWS Marketplace
AWS Quick Starts
AWS Support

**Vínculos relacionados**

Documentos técnicos de AWS

# Some Web Service URLs



- Apache Axis (Java and C++)
  - http://xml.apache.org/axis/
- NaradaBrokering
  - Java support for Reliability, Eventing, etc.
- WS/XSUL from Indiana University Extreme Labs
  - http://www.extreme.indiana.edu/xgws/xsul/index.html
- gSOAP: C++ SOAP toolkit
  - http://https://www.genivia.com/dev.html
- Python Web Services:
  - http://pywebsvcs.sourceforge.net/

# References



- https://realpython.com/api-integration-in-python/#rest-architecture
- Django documents information
  - https://www.djangoproject.com/
  - https://docs.djangoproject.com/en/5.2/
- Flask information
  - https://flask.palletsprojects.com/en/stable/

# INTRODUCTION TO WEB SERVICES



BY NC SA
for the Alejandro Calderón Mateos work